Empório da Música

Agente de atendimento por texto para a Empório da Música, loja de instrumentos em Campo Grande/MS. Responde sobre produtos, preços, promoções, pagamento, pedidos, trocas e regras da loja, e registra reclamações.

A regra central do projeto: preço, estoque, promoção, parcelas, prazos e dados de pedido vêm sempre de código determinístico. O modelo decide o que consultar e redige a resposta, mas não calcula nada.

Como executar

Requer Python 3.11+, uv e uma chave do OpenRouter.

```
uv sync
cp .env.example .env    # preencha OPENROUTER_API_KEY
uv run emporio --data 2026-02-20
```

Digite `sair` para encerrar. Use `--debug` para ver as ferramentas chamadas, com argumentos e duração; e-mail e telefone aparecem mascarados.

Por que `--data 2026-02-20`

Os dados vão até março de 2026. Com a data de hoje, todos os pedidos entregues estão fora dos prazos de troca. Com a data fixa, o pedido 7 está no prazo e o 6 não, o que permite ver as duas respostas. Sem `--data`, vale a data atual em Campo Grande.

Mensagens para testar

Mensagem	O que observar
`Quais violões vocês têm até R$ 1000?`	Só disponíveis, por menor preço, com o total encontrado
`Quanto custa o Takamine GD20?` e depois `e no PIX?`	Pergunta de seguimento sem repetir o produto
`Posso parcelar o Yamaha C40 em 12x?`	Indica o máximo permitido, 6x
`Qual o status do pedido 8?`	Pede e-mail ou telefone antes de mostrar qualquer dado
`Me arrependi da compra, posso devolver?`	Pede pedido e contato, depois aplica o prazo
`Vocês vendem cordas de guitarra?`	Redireciona, porque a loja vende só instrumentos

Conversas completas em `examples/`.

Reclamações

```
uv run emporio reclamacoes
```

Lista as reclamações registradas, da mais recente para a mais antiga. O arquivo fica em `data/runtime/`, fora do Git, porque tem dados pessoais.

API opcional

```
uv sync --extra api
uv run --extra api uvicorn emporio.api:app
```

Com o servidor no ar, em outro terminal:

```
curl -s -X POST 127.0.0.1:8000/chat -H 'content-type: application/json' \
  -d '{"mensagem":"Qual o horário da loja?"}'
```

Devolve `session_id`, que deve ser reenviado para continuar a conversa. Não tem autenticação: use só em `127.0.0.1`. Conversas feitas pela API também aparecem no dashboard.

Dashboard de demonstração

```
uv sync --extra api
EMPORIO_DATA_REFERENCIA=2026-02-20 uv run --extra api uvicorn emporio.api:app
```

Abra http://127.0.0.1:8000/. A página mostra o chat, o histórico de conversas, as consultas e o tempo de cada resposta, além de alguns números. A variável fixa a data, pelo mesmo motivo do `--data` na CLI.

O histórico fica em `data/runtime/conversas/`, um JSON por conversa, fora do Git. Depois de reiniciar o servidor, as conversas continuam visíveis, mas somente leitura: as confirmações de identidade não são restauradas, então nenhum dado de pedido reabre sem nova confirmação.

Testes

```
uv run pytest -m "not e2e"    # 360 testes, sem rede e sem custo
uv run pytest -m e2e          # 99 cenários com o modelo real; cerca de US$ 0,16 por rodada
```

Arquitetura

```
Cliente → CLI, API ou dashboard → Agent (laço de tool use) ⇄ OpenRouter
                          │
                          ├─ buscar_produtos, detalhes_produto — Repository — CSVs
                          ├─ consultar_pedido ──────────────────── SessionState
                          ├─ consultar_politica ────────────────── manual em markdown
                          └─ registrar_reclamacao ──────────────── data/runtime/
```

Arquivo	Responsabilidade
`src/emporio/agent.py`	Laço de tool use, janela do histórico e falhas do LLM
`src/emporio/tools.py`	Ferramentas e schemas, gerados dos modelos Pydantic
`src/emporio/rules.py`	Regras puras: PIX, parcelas, prazos e identidade
`src/emporio/data/`	Carga, validação e consultas aos CSVs
`src/emporio/session.py`	Histórico e confirmações, fora do alcance do modelo
`src/emporio/historico.py`	Conversas salvas e indicadores do dashboard
`src/emporio/static/dashboard.html`	A página inteira, sem build nem dependências
`src/emporio/policies.py`	Seções do manual
`src/emporio/prompts.py`	System prompt
`src/emporio/complaints.py`	Registro de reclamações

Ferramentas

O modelo escolhe a ferramenta e os argumentos. O código valida os argumentos com Pydantic, executa a consulta e devolve JSON.

Ferramenta	Recebe	Devolve
`buscar_produtos`	Categoria, texto, faixa de preço ou só promoções	Produtos disponíveis, com o total encontrado e o preço pronto para exibir
`detalhes_produto`	Nome, mesmo aproximado, ou id	Preço, situação, PIX, parcelas e alternativas; se houver dúvida, os candidatos
`consultar_pedido`	Número do pedido e, se o cliente informou, o contato	Status, rastreio, itens e prazos de troca, só depois de confirmar a identidade
`consultar_politica`	Uma das 9 seções do manual	O texto da seção
`registrar_reclamacao`	Resumo e contato; nome e pedido são opcionais	O número de protocolo

Erros não encerram o turno: argumento inválido ou falha na consulta volta ao modelo como `{"erro": ...}`, sem detalhes internos. Quando a resposta exige cuidado, como em produto não encontrado ou identidade não confirmada, o campo `orientacao` diz ao modelo o que falar.

Decisões técnicas

- **OpenRouter:** dá acesso a vários modelos pela mesma API. Foi o que permitiu comparar modelos só trocando uma variável de ambiente.
- **Function calling, em vez de RAG ou text-to-SQL:** as perguntas são sobre dados estruturados com regras de negócio. Ferramentas tipadas deixam o cálculo em Python, onde ele é testável, e impedem consultas livres aos dados.

- **Laço escrito à mão, sem LangChain ou LangGraph:** são poucas dezenas de linhas, fáceis de ler e de testar com um cliente falso. Um agente único com cinco ferramentas não precisa de grafo.
- **Manual por seção, em vez de busca vetorial:** são 9 seções curtas. O modelo escolhe o tópico por um `enum`, sem índice nem embeddings, e a resposta é sempre um texto conferido.
- **Dinheiro em `Decimal`:** os CSVs são lidos como texto, para não passar por `float`.
- **Identidade aplicada em código:** o modelo não consegue pular a confirmação, por mais que o cliente insista. O contato precisa ser do dono do pedido e ter sido escrito pelo cliente, e vale só para aquele pedido e sessão. Pedido inexistente e dado errado têm a mesma resposta, e a terceira falha bloqueia novas confirmações.
- **A ferramenta entrega o que o modelo erraria:** testes reais mostraram que regras no prompt não bastam. `preco_exibicao`, `prazos`, `parcelas_sem_juros` e `orientacao` trazem o texto ou a conclusão prontos.
- **Protegido contra promessa sem consulta:** se o modelo diz "vou verificar" sem chamar nada, o laço repete a chamada uma vez, exigindo uma ferramenta.
- **Limite total de 60 s por chamada ao LLM:** o `timeout` do cliente vale por leitura, e as tentativas se somam. Se estourar, o cliente recebe a mensagem de contingência com o telefone da loja.

Escolha do modelo

O padrão é `anthropic/claude-haiku-5.5`, trocável por `OPENROUTER_MODEL`. O modelo precisa aceitar `tools` e `tool_choice`.

Comparação feita durante o desenvolvimento, com a suíte de 18 cenários que existia na época, em 3 rodadas:

Modelo	Aprovados	Preço por milhão de tokens (entrada/saída)
`anthropic/claude-haiku-5.5`	54 de 54	US$ 0,10 / 0,50
`openai/gpt-6-luna`	53 de 54	US$ 0,10 / 0,50
`qwen/qwen3.5-9b`	51 de 54	US$ 0,17 / 0,25
`google/gemma-4-26b-a4b-it`	46 de 54	US$ 0,07 / 0,23

Dos 13 modelos gratuitos testados, 11 não responderam: limite do provedor, indisponibilidade, restrição de uso ou bloqueio pela configuração de privacidade da conta. Os outros dois fizeram 6 de 8 em uma amostra menor. O `claude-sonnet-5.5`, previsto no plano, não foi medido, por limite de crédito.

Resultados com o modelo padrão

Medidos em 2 rodadas seguidas da suíte final, com `claude-haiku-5.5` e a data fixa:

Critério	Meta	Resultado
Cenários `e2e` aprovados	100%	99 de 99, nas 2 rodadas
Tempo de resposta, percentil 95	Até 10 s	4,2 s e 4,7 s
Resolvidas sem a equipe	Pelo menos 80%	87% e 90%
Custo	Não definida	Cerca de US$ 0,16 por rodada

Suposições sobre os dados

- **Promoções não têm datas.** Ativa é `is_active = 1`. Se houver várias ativas, vale o maior desconto. Encerrada é `is_active = 0`, e entre várias vale o maior `promotion_id`.
- **Não há data de recebimento.** A previsão de entrega é usada como aproximação, e arrependimento, troca por preferência e garantia legal contam dela.
- **O prazo de defeito conta da data da compra,** como diz o manual.
- **O último dia do prazo está dentro do prazo.**
- `status` **do produto prevalece sobre o estoque:** descontinuado com estoque continua fora do catálogo.
- **Telefone:** o manual traz dois números de WhatsApp. O agente usa (67) 3341-4444, dos dados da empresa.
- **Parcelas:** o manual é ambíguo em "até 3x, sem valor mínimo (exceto abaixo de R$ 50,00)". Foi interpretado como 1x sem mínimo e 2x ou 3x com mínimo de R$ 50.
- **Catálogo:** o manual cita mais de 300 instrumentos, mas o catálogo tem 65. O agente responde só com esses.

Limitações conhecidas

- Um modelo escrito errado pode ser tratado como o produto mais próximo: `Rozini R 104` traz o preço do RC-104, sem avisar da diferença, e o agente pode falar do modelo citado como se ele existisse.
- Os itens de pedido não trazem o preço pago, então a resposta mostra só nome e quantidade.
- Os totais dos pedidos 3 e 20 não batem com a soma dos itens pelo preço atual. O agente informa o total registrado.
- Os pedidos 9, 16 e 18 foram parcelados abaixo da parcela mínima do manual. O agente mostra o pagamento registrado, sem apontar isso.
- O bloqueio de identidade é por sessão: uma sessão nova recomeça a contagem. Em produção, ele precisaria valer também por pedido ou por origem.
- A API e o dashboard não têm autenticação, e as sessões ficam em memória, sem limite nem expiração.

- O histórico do dashboard guarda as mensagens como foram escritas, inclusive e-mail e telefone digitados pelo cliente, sem limite de retenção. A máscara vale só para os parâmetros das consultas.
- O dashboard formata parágrafos, listas e negrito. Tabelas e links em Markdown aparecem como texto.
- O histórico da conversa, incluindo o contato escrito pelo cliente e o resultado das consultas, é enviado ao provedor do modelo. O uso em produção exige um provedor com contrato adequado à LGPD.
- O agente não sabe de feriados. Fora do horário, a volta da equipe é calculada pelo modelo a partir do manual.
- Uma execução anterior de teste com um modelo gratuito travou por 51 minutos, sem causa confirmada. O limite de 60 s por chamada foi criado para esse risco, mas a causa não foi reproduzida.
- A proteção contra promessa sem consulta depende de uma expressão regular com frases em português, e pode não reconhecer outras formas.
- O tom ainda não foi avaliado por uma pessoa da loja (SC-010), e o README não foi testado por alguém de fora (SC-011).

Próximos passos

- SQLite no lugar dos CSVs, sem mexer nas ferramentas, graças à camada de repositório.
- Persistir sessões e bloqueios.
- Integração com WhatsApp.
- Busca vetorial, se o manual crescer.
- Avaliação automática com um modelo como juiz, para medir tom.
- Validar a resposta em código antes de entregá-la, por exemplo conferindo valores em reais.

Uso de assistentes de IA

O projeto foi construído com o Claude Code, seguindo o fluxo do Spec Kit: especificação, plano, tarefas e implementação, em `specs/001-agente-atendimento/` para o agente e em `specs/002-dashboard-demo/` para o dashboard. A pessoa responsável definiu os requisitos, a stack e as prioridades, e decidiu o modelo e o escopo em cada etapa. O assistente escreveu o código, os testes e a documentação.

Os dados corrigiram o desenho inicial em vários pontos, como promoções sem data, defeito contado da compra e mínimo de parcela em 2x e 3x. Os testes com modelo real revelaram falhas que os testes sem rede não pegam, como prometer uma consulta sem fazê-la.